

One Namespace, or One **Vote** Each

Fourteen cards from nine runnable topologies. Cards 01 to 06 build the words, simple first: the map, Raft, placement, accounts and domains. Cards 07 to 12 use those words on each shape. Card 13 tests one procedure for moving between two of them. Card 14 asks where T4's meta leader should live, and whether it stays there.

THE QUESTION EVERY CARD SERVES

Two regions, one platform. **Do you want one global stream name, or do you want each region to keep working when the other goes dark?** With two sites, no shape in this lab gives both. A third site that votes keeps one meta group alive when either region is dark (card 08). A stream stored in the dark region is still dark with it.

FOUR SENTENCES THE DECK DEPENDS ON

A **gateway** joins clusters into one supercluster with **one meta group**. A **leaf link**, as this lab builds it — `$SYS` not carried, a domain per side — leaves a meta group on each side. An **account** is the wall for data. A **domain** names a JetStream, and on a `$SYS` leaf it can stop JetStream extending across the link; it is still not a security boundary.

CARD FAMILIES

- Map — the topologies and the words
- Placement — where a stream lands
- Links — gateway, leaf node
- Traps — both links at once

- Quorum — Raft, three groups, arbiters
- Boundaries — accounts, domains
- Copying — mirrors, export / import
- Moving — switching shapes in place

1

KEY

Every card is drawn to the report's rules

The figures on these cards use the **same drawing grammar as `REPORT.html`**. A boundary means the same thing in both. If a card and the report disagree, the report is right — it is generated from the run.

BOUNDARIES AND LINKS

JETSTREAM DOMAIN — A NAME

ACCOUNT — THE WALL FOR DATA

orange dashed — jetstream.domain. Names one JetStream and sets its API prefix `$JS.<domain>.API`. Not a wall. It does not create a meta group by itself (card 06).

ACCOUNT LB_ZA

CLUSTER za — REGION za

yellow dashed — an account. Two accounts cannot see each other's subjects, streams or consumers without an export and an import. plain box = a cluster. red node = frozen.

gateway — one supercluster, one meta group

leaf — in this lab, a meta group each side

mirror — a copy somebody wrote down

kill -STOP — how every outage here is made

Each figure ends with the check IDs it draws. A claim with no ID carries a tag: (documented), (inferred) or (proposed).

FOUR KINDS OF STATEMENT — EACH ONE IS TAGGED

TAG	WHAT IT MEANS
A11 MEASURED	This rig ran it and the answer matched. The check ID points at one row of <code>REPORT.md</code> . An ID with a small letter, such as <code>A11a</code> , is a recorded observation, not a pass or fail.
DOCUMENTED	The NATS documentation says it. This lab did not test it. The source is on the provenance page.
INFERRED	It follows from measured checks and the documented rules, but no check asks it directly.
PROPOSED	A plan, such as a migration step. Nobody has run it.

TWO THINGS THE DRAWINGS WILL NOT LET YOU ASSUME

> A domain box is not an account box. They are different colours because they are different kinds of boundary. A domain separates *names*. An account separates *data*, and nothing crosses it without an export.

> A frozen region is `kill -STOP`, not a severed link. In a real WAN partition both sides stay alive and cannot reach each other. The lab did not build that rig. Where a card draws a live cut, the cut is tagged

INFERRED

.

01

MAP — THE NINE TOPOLOGIES

Five shapes, four variants, one letter each

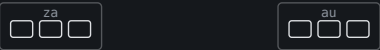
Demo 03 built **nine runnable topologies** from nothing and measured each one: five shapes, **T1 to T5**, and four variants, **B, E, G and H**. G is also called **T6**. Every check ID starts with the letter of the rig that ran it.

DECISION

Read a check ID as a place: **A11** is check 11 on rig A, which is T2. Before you trust a claim, find its letter here and look at the shape it was measured on.

THE SIX DRAWN SHAPES — SERVER COUNT AND META GROUPS ARE MEASURED

T1 — two islands
6 SERVERS · 2 META GROUPS · checks T1a...



no link at all — the control

T2 — one gateway
6 SERVERS · 1 META GROUP · checks A...



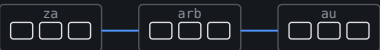
one supercluster over two regions

T3 — gateway + 1-node arbiter
7 SERVERS · 1 META GROUP · checks C...



a third site with one server

T4 — gateway + 3-node arbiter
9 SERVERS · 1 META GROUP · checks F...



a third site with three servers

T5 — hub and leaf
9 SERVERS · 3 META GROUPS · checks D...



a domain per site; \$SYS not carried

T6 — gateway and hub leaf
9 SERVERS · 2 META GROUPS · checks G...



both links at once (variant G)

THE FOUR VARIANTS — EACH CHANGES ONE THING AND ASKS ONE QUESTION

VARIANT	WHAT IT CHANGES	CARD
B · on T2	A different <code>jetstream.domain</code> on each gateway-joined cluster. Does a domain split the meta group?	06
E · on T2	Two accounts, and an export / import between them. Can a region share data on purpose, across the account wall?	11
G · T6	A gateway and a hub leaf link. Runs twice: phase 1 with no domain, phase 2 with a domain per site.	12
H · on T5	An account per region carried over the leaf link, beside the shared account. Does that stop double capture?	09

T1 is the control. Two clusters, no link, a meta group each. Every other shape adds one link and is compared with it.

MECHANISM — THE WORDS, SIMPLE FIRST

WORD	WHAT IT MEANS	IN THIS LAB
Server	One <code>nats-server</code> process.	6, 7 or 9 per rig, all on one laptop.
Cluster	Servers joined by routes. They act as one.	One per region: <code>za</code> , <code>au</code> . Plus <code>arb</code> or <code>hub</code> .
Gateway	Joins clusters into one supercluster .	T2, T3, T4, T6.
Leaf link	A server or cluster connects to another as a leaf. Each remote binds one account.	T5, H, T6. Binds <code>LB</code> (H adds <code>LB_ZA</code> / <code>LB_AU</code>). Never <code>\$SYS</code> .
Account	Its own subjects, streams and consumers. The wall for data.	<code>LB</code> shared; <code>LB_ZA</code> , <code>LB_AU</code> per region; <code>\$SYS</code> the system account.
JetStream	The storage layer: streams, consumers, KV. One JetStream is the set of servers that share one meta group.	Enabled on every server.
Meta group	The Raft group of every server in one JetStream. It holds the catalogue — which streams and consumers exist, and where. Its leader handles create, update and delete.	1 on T2–T4, 2 on T1 and T6, 3 on T5 and H.
Domain	<code>jetstream.domain</code> : a name for one JetStream, and its API prefix <code>\$JS.<domain>.API</code> .	None on T1–T4 and G phase 1. <code>hub</code> / <code>za</code> / <code>au</code> on T5, H, G phase 2. B: one per cluster.
JetStream namespace	Where a stream name must be unique: one meta group × one account .	Card 05.

HELD STILL ON EVERY RIG

NOT TESTED ON ANY RIG

› `nats-server` 2.14.6 and the `nats` CLI 0.4.0, on one host.

› JetStream on every server. Accounts are user / password pairs in hand-written `.conf` files. No operator trust chain.

› `$SYS` is never carried over a leaf link. This one fact shapes cards 06, 09 and 12.

› Every outage is `kill -STOP` on whole servers, proved dark before it is measured.

› A real WAN: latency, packet loss, a live two-sided partition.

› A leaf link that **carries** `$SYS` . The docs say it changes the result `DOCUMENTED` . Card 06.

› A real operator (`nsc`) trust chain, or JWT accounts.

› Mirror lag, throughput and any production timing.

VERDICT

Nine words carry the whole deck. The next five cards use them, one idea at a time.

T1a–T1c, A1, C1, F1, D1, G1 · Card 01

02

QUORUM — RAFT

A vote needs more than half, and servers vote, not sites

JetStream keeps its state in **Raft groups**. A Raft group is a set of servers that elect one **leader**. The leader can act only while **more than half** of the group can hear it. That number is the **quorum**, or majority. There is **more than one kind of group**, and **this card counts the meta group**.

DECISION

Before you pick a shape, count the servers in the **meta group**. Ask: **when one site goes dark, is more than half of it still alive?** If not, it stops. Sites do not vote; servers do.

THREE KINDS OF RAFT GROUP — EACH ELECTS ITS OWN LEADER AND COUNTS ITS OWN MAJORITY

GROUP	HOW MANY	ITS SERVERS	WHAT STOPS WHEN IT LOSES ITS MAJORITY
Meta group — this card	one per JetStream	every server in that JetStream	create, update and delete of streams and consumers (A11)
Stream group	one per stream	its 1, 3 or 5 replicas, in one cluster (F17)	storing and reading that one stream (A21)
Consumer group	one per consumer	the servers of its stream (A18)	delivery and acks for that one consumer DOCUMENTED

>

The meta leader is not a stream leader. Each group holds its own vote [DOCUMENTED](#). With no meta leader, a publish into an existing stream is stored only while that stream's own group keeps its majority ([A12](#) , [F21](#)). Card 03 asks all three questions.

META GROUP MAJORITY, THEN A DARK SITE — GREEN IS LIVE, RED IS DARK, GREY IS THE REST

A MAJORITY IS MORE THAN HALF: FLOOR(N / 2) + 1 (DOCUMENTED)

3 servers

majority 2 — T1b T1c D2

6 servers

majority 4 — A2

7 servers

majority 4 — C1

9 servers

majority 5 — F1

A SITE GOES DARK — COUNT THE SERVERS THAT ARE LEFT

T2 · ZA dark

3 of 6, below 4 — no leader — A10

T4 · AU dark

6 of 9 — a leader, creates — F3 F4

T4 · AU + 1 arb

5 of 9, exactly 5 — still elects — F5 F6

T4 · 4 of 9 left

4 of 9 — no leader, create fails — F7 F8

T5 · AU dark

3 groups of 3 — ZA counts alone, 3 of 3 — D24 D25

a live WAN cut (inferred, not built): a 3 | 3 split leaves no side a majority of 6.

a 6 | 3 split leaves the side of 6 a majority of 9. two sides can never both hold a majority of one group.

An even split of an even group stops both halves. Three sites break the tie; two sites cannot.

MECHANISM

>

Raft keeps one ordered log per group. The leader adds an entry; the entry is **committed** when a majority has stored it. The docs give quorum as half the cluster size, plus one [DOCUMENTED](#).

>

A majority is always **more than half**. So two groups of servers that cannot reach each other can never both hold a majority of the **same** group. One side, at most, keeps a leader.

>

An **uneven** split can leave one side a majority: 6 of 9 keeps a leader ([F3](#)). An **even** split of an even group leaves no side one: 3 of 6 has no leader ([A10](#)).

>

Every dark site in this lab is [kill -STOP](#). A live WAN cut, where both sides run and cannot talk, uses the same arithmetic, but it was not built [INFERRED](#).

SLACK — WHAT EACH SHAPE CAN STILL LOSE AFTER ONE REGION IS DARK

SHAPE	SERVERS IN THE META GROUP	MAJORITY	LEFT, ONE REGION DARK	SLACK
T1, T5 each site	3	2	3 — the other site is not in this group T1g D24	1 server INFERRED
T2 gateway	6	4	3 — no leader	none A10
T3 + 1-node arbiter	7	4	4 — exactly the majority	none C12
T4 + 3-node arbiter	9	5	6	1 server F11 F5
T6 ZA + AU group	6	4	3 — no leader	none G10

WHAT IT DOES GIVE YOU

>

The arithmetic is exact. Count the live servers in a group and you know if it can act. Every leader and no-leader result above was measured.

>

At most one side keeps a leader. Raft does not let two leaders change the same group.

WHAT IT DOES NOT GIVE YOU

>

A group that spans two sites stops when either site goes dark, if the sites are equal.

>

The [leader](#) field in [/jsz?meta=1](#) stays stale for about a minute after a majority dies. The rig polls for it; a person reading it once will be misled.

>

A named leader is not readiness. [/jsz](#) can name a new leader before management calls succeed; a create asked at once was refused ([F6a](#)). Call it ready only when a real operation, such as a stream create, succeeds. Retry, and record the wait.

VERDICT

Put the majority where the work must go on. Card 03 shows that there is more than one group to count.

A2, A10, C1, C12, F1–F11, D2, D24 · Card 02

03

QUORUM — THREE KINDS OF RAFT GROUP

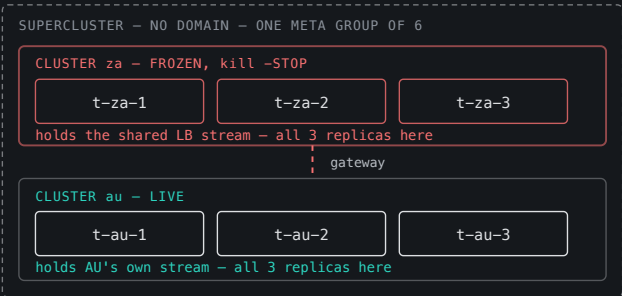
"Can we still write?" is three questions

Card 02 counted the meta group. The other two kinds, **one group per stream and one per consumer**, count their own majority. So in a gateway shape with a region dark, **one question fails and the others can still work**.

DECISION

Before you accept any topology, say **which of the three you need during an outage**. Losing the meta group is a **change** freeze, not a **write** freeze. They are not the same outage.

ONE RIG, THREE ANSWERS — T2 GATEWAY, ZA FROZEN, AU LIVE



- 1 — CREATE A STREAM · the META group needs a majority of all 6 servers; 3 of 6 is below 4 fails, 10008 — A11 A11a
- 2 — PUBLISH TO AU'S STREAM · the STREAM group needs only that stream's own 3 replicas, all in au 1 message stored — A12
- 3 — READ · the stream, through a CONSUMER AU's own stream: ok — A20 the shared stream in za: fails, 10008 — A21 A21a a consumer lives with its stream — A18 A19

a plain nats pub is a CORE publish. it exits 0 whether the message was stored or not. the only proof is counting the stream afterwards — A12 F21 D23 G12 H16

The same drawing, three different answers. **The shape is not "up" or "down".**

MECHANISM — THREE KINDS OF GROUP, THREE MAJORITIES

GROUP	HOLDS	ITS SERVERS
Meta — one	The catalogue: which streams and consumers exist, and where. Create, update and delete go through it.	Every server in one JetStream. 6 on T2.
Stream — one each	The messages of one stream. A publish is stored when a majority of its replicas has it.	Its replicas: 1, 3 or 5 DOCUMENTED , all in one cluster (F17).
Consumer — one each	One consumer's position: what was delivered and acked.	With its stream A18 A19 .

- › The docs name the three kinds and say each stream and each consumer is its own Raft group **DOCUMENTED**.
- › A read of the dark region's stream fails and a read of the live region's stream works, on the same rig, at the same moment (A20 , A21). So the read follows the stream's own replicas, not the meta group. T4 proves it: there the meta group survives a dark ZA, and the read of ZA's stream still fails (F28). On T2 the error is 10008 (A21a); on T4, with the vote alive, it is a timeout (F28a).
- › This split is why D03–R1 in the report is three columns, not one.

THE THREE QUESTIONS AGAINST EVERY SHAPE MEASURED

SHAPE	CREATE, WITH A REGION DARK	PUBLISH INTO A LOCAL STREAM	READ, WITH A REGION DARK
T1 no link	yes T1g T1h	not measured	not measured
T2 gateway	no — 10008 A11	yes A12	own: yes A20 dark region's: no A21
T3 + 1-node arbiter	yes C4 C6	not measured	not measured
T4 + 3-node arbiter	yes F4 F23	yes F21 F24	own: yes F27 dark region's: no F28
T5 hub and leaf	yes D25	yes D26	not measured
H T5, account per region	not run	not run	not run
T6 both links at once	no — 10008 G11	yes G12	not measured

WHAT ASKING IT THREE WAYS DOES GIVE YOU

- › You learn that a gateway outage is a **change freeze**, not a full stop.
- › You can size the risk properly: creation stops, existing traffic in the live region carries on.
- › It tells you exactly which checks to demand of any claim somebody makes.

WHAT IT DOES NOT GIVE YOU — THE CATCH IN ROW TWO

- › "Publish keeps working" holds on **one condition**: that stream's own replicas must still hold their own majority.
- › Every outage here darkens a whole region, so the surviving streams kept all three replicas at home.
- › Losing **all** of one stream's replicas while the vote lives is measured: that stream cannot be read (F28).
Losing only some of them is a different question, and is not measured here.

VERDICT

Write the outage sentence down before choosing a shape: **"during a region outage we must still be able to <create / publish / read>."**

04

PLACEMENT — WHERE A STREAM LANDS

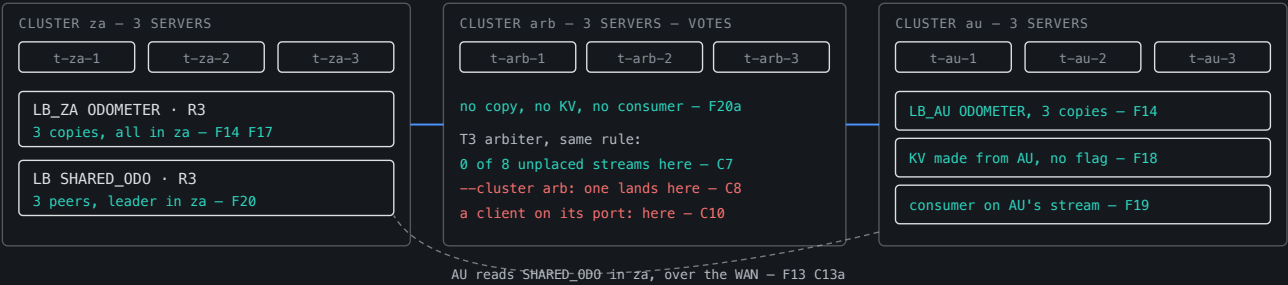
Placement decides where bytes sit, not who survives

Every stream is placed **once, when it is created**, into **one cluster**. Ask for nothing and it lands where the client is. Name a cluster and it lands there. T4 is the reference here: three clusters, one meta group of 9.

DECISION

For every stream, KV bucket and consumer, **say which cluster it belongs in**, and make the create call say so. A stream stored in the dark region is dark with it, whatever the quorum does.

T4 — WHAT LANDED WHERE, WITH NO REGION DARK



PLACEMENT IS PER STREAM, AT CREATE TIME

no flag → the cluster of the server the client is connected to — F18 A9 C18
—cluster X → cluster X, from any client — E13 E14 on T2 — on T4 (inferred)
every replica of one stream sits in one cluster — F17 C17
that cluster must hold R servers, or the create fails with 10005 — C9 B4

The arbiter votes and stores nothing — until somebody places a stream on it.

MECHANISM

- › **Placement belongs to a stream**, not to an account. Two streams in one account can sit in two clusters: `--cluster za` and `--cluster au` from the same client both work (E13 , E14).
- › This lab set the **initial placement**, at create time. A stream config can name a cluster or tags `DOCUMENTED`. This lab tested `--cluster` only; tags were not tested. NATS also documents changing placement later, which moves the replicas `DOCUMENTED` (docs: placement). **Moving a stream was not tested here.**
- › With no placement, the stream lands in the client's own cluster. The meta leader records the choice; **where the meta leader sits does not pick the cluster** `INFERRED` — no check moved the leader to test it.
- › **Placement is not quorum.** T4 survives a dark region because of the meta majority (card 02). A stream placed in that dark region is still dark.
- › **Control path and data path differ.** A read from AU of a stream in za gives `1 / 1` (F13 , A26): that is **one** stream, read over the WAN (C13a), not two copies. A second copy in au needs a mirror (A15 , card 10).

WHAT YOU ASK, AND WHERE IT LANDS

YOU ASK	LANDS IN	PROOF
no flag, from a ZA client	za — never the arbiter	C7 (T3, 8 streams)
no flag, KV bucket from an AU client	au	F18 A9 C18
<code>--cluster za</code> / <code>au</code> , same client	as named	E13 E14 (T2) · T4 <code>INFERRED</code>
a mirror, placed in the other region	au	A15
<code>--cluster arb</code> , R1	arb — unreplicated	C8 (T3)
any create from a client on the arbiter's port	arb	C10 (T3)
<code>--cluster arb</code> , R3, one-node arbiter	fails, 10005	C9
the blind cluster of variant B	fails, 10005	B4 , card 06

WHAT IT DOES GIVE YOU

- › **Predictable.** Local by default, explicit when named. Every row above landed where the rule says.
- › Replicas stay in one cluster (F17), so a dark region never holds half a stream.

WHAT IT DOES NOT GIVE YOU

- › **A shared-account projection can split.** Its consumer lands with a ZA stream while its KV, made from AU, lands in au (A19 , F18).
- › A consumer on a far stream pays the WAN on every pull (A19).
- › The arbiter stores whatever is placed on it (C8 , C10). A 3-node arbiter could even take an R3 stream `INFERRED`.

VERDICT

Name the cluster in every create call, and keep clients off the arbiter's port (C11).

05

BOUNDARIES — ACCOUNTS AND THE JETSTREAM NAMESPACE

A stream name is unique in one meta group and one account

Two things decide whether two regions can both own a stream called `ODOMETER` : how many meta groups there are, and whether they share an account. `One meta group × one account` is one JetStream namespace. Only there does a name clash.

DECISION

Give each region **its own account** unless you truly want one name for both regions. Then pick the link. The account choice works the same on every shape; the link choice does not.

TWO CHOICES, FOUR OUTCOMES — ONE PUBLISH IN ZA, THEN COUNT

ONE META GROUP
joined by a gateway
T2, T3, T4, and
T6's two regions

SHARED ACCOUNT LB — BOTH REGIONS

LB — ONE NAMESPACE

za · ODOMETER — au · the same name

a 2nd ODOMETER is refused, 10058 — A4 B8 G4
1 / 1 = ONE copy, read from both sides — A26 G7a
AU's stream info hands back za's stream — A5

AN ACCOUNT PER REGION — LB_ZA, LB_AU

LB_ZA — LB_AU

za · ODOMETER — au · ODOMETER

each creates its own ODOMETER — A6 A7 E7 E8
1 / 0 — the other account sees nothing — A8
two namespaces on one meta group

A META GROUP
PER SITE
no link, or this
lab's leaf link
T1, T5, H

LB — A NAMESPACE PER SITE

za · ODOMETER — au · ODOMETER

both create ODOMETER, no 10058 — T1d D3
T1, no link: 1 / 0 — T1f
T5, LB over the leaf: 1 / 1 STORED, TWO copies — D5 D5a

LB_ZA — LB_AU

za · ODOMETER — au · ODOMETER

1 / 0 — nothing is shared — D18 H6
a shared hub is not a shared account — H6a

ONE META GROUP × ONE ACCOUNT = ONE JETSTREAM NAMESPACE. A NAME IS UNIQUE ONLY INSIDE ONE.

Same number, opposite meaning. 1 / 1 over a gateway is one copy. 1 / 1 over this lab's leaf is two.

MECHANISM

› An **account** owns its subjects, streams and consumers. Nothing crosses to another account without an export and an import (card 11).

› A **meta group** keeps one catalogue. Inside it, one account can hold one stream per name. So the unit where a name must be unique is **one meta group × one account**: the JetStream namespace.

› A **domain name does not split it**. Variant B put two domain names on one meta group and still got `10058` (`B8`). Card 06.

› **The meta group is not per account**. T2 has one meta group with three JetStream accounts on (`A1`). So a dark region freezes creates in every account at the same moment `INFERRED`. An account is a wall for data, not for availability.

› On T5 the leaf binds `LB` , so a subject crosses the link, and **each** meta group's `ODOMETER` stores the publish (`D5`). Neither can refuse the other's name: two same-named shadow streams hold `1 / 2` (`D6`).

COUNT	WHERE	WHAT IT MEANS
1 / 1	one meta group, shared account — <code>A26 C13 F13 G7</code>	One stored copy, read from both sides. The far read crosses the WAN (<code>C13a</code>).
1 / 1	a meta group per site, shared account over the leaf — <code>D5</code>	Two stored copies of one send. Double capture (<code>D5a</code>).
1 / 0	an account per region, any shape — <code>A8 D18 H6</code>	The other region's account never sees it.
1 / 0	T1, shared account, no link — <code>T1f</code>	Nothing connects the two sites at all.

WHAT IT DOES GIVE YOU

› An account per region behaves the same on every shape. Own names, own data, 1 / 0 (`A6 A7` , `D18` , `H6`).

› A shared account over a gateway gives one global name and stores one copy (`A4` , `G7a`).

WHAT IT DOES NOT GIVE YOU

› A shared account over this lab's leaf captures twice, silently (`D5` , `D6`).

› Over a gateway, AU's `stream info ODOMETER` returns za's stream (`A5`). It looks local. It is not.

› Per-region accounts do not share data. Sharing needs a mirror or an export (cards 10, 11).

VERDICT

The account is the wall. The meta group decides how big the room is.

A4–A8, D3–D6, D18, H6 · Card 05

06

BOUNDARIES — DOMAINS AND LEAF LINKS

A domain names a JetStream. It is not a wall.

`jetstream.domain` gives one JetStream a name and an API prefix, `$JS.<domain>.API` . It does **not** decide which servers vote together: the links and the **system account** (`$SYS`) decide that. A domain string alone does not make a meta group.

DECISION

Inside one supercluster, use **one domain name everywhere, or none**. Across a leaf link, a domain per side is how you name, and reach, each JetStream. **Never use a domain as a security boundary**.

THREE RIGS — WHAT THE DOMAIN DID, AND WHAT THE LINK DID

B · GATEWAY + TWO DOMAIN NAMES
INVALID — B6 G19–G22

DOMAIN za

DOMAIN au

still ONE meta group of 6 — B3
only 1 of 2 sides elects — B1 B2
place on the blind side: 10005 — B4
which side goes blind: a coin toss — G19a

T5 / H · LEAF, A DOMAIN EACH
\$SYS NOT CARRIED OVER THE LEAF

hub · domain hub

za · domain za

au · domain au

3 meta groups — D1 H1
a stream name alone: fails — H7
\$JS.<domain>.API: crosses — D8 H11

G PHASE 1 · LEAF, NO DOMAIN
\$SYS NOT CARRIED OVER THE LEAF

hub · no domain

za · no domain

au · no domain

2 meta groups, 3 / 6 / 6 — G1 G2
the hub stays its own JetStream
cause: \$SYS not carried (documented)

DOCUMENTED IN THE NATS DOCS — NOT TESTED ON THIS RIG

a leaf that carries \$SYS and sets no domain, to a clustered hub, extends the hub's JetStream: one meta group.

a domain on the leaf side stops that extension. a domain may change across a leaf, never inside a supercluster.

one domain name on every server of a supercluster, or none, is valid.

a domain names a JetStream and its API. the links and \$SYS decide which servers vote together.

An account (the yellow box on card 05) stops data. **A domain (the orange box here) names a JetStream. It is not a wall**. Neither box decides who votes.

MECHANISM

- Routes and gateways join servers into one JetStream with one meta group. A leaf link joins the JetStream on each side **only** when it carries the system account and neither side sets a different domain [DOCUMENTED](#) ([docs: leaf nodes](#)).
- This lab never carries `$SYS` over a leaf. With a domain per side, that gave three JetStreams (`D1` , `H1`). With no domain at all, the hub still stayed separate (`G1` , `G2`). So in this lab the missing `$SYS` link is enough on its own [INFERRED](#).
- A stream name resolves **in the local domain** (`H7a`). To reach another JetStream, name its API: `external.api = $JS.za.API` (`D8` , `H11`). `external.api` is a JetStream API prefix. In this lab the prefix names a domain, and binding an account per region changed nothing (`H11a`). The prefix can also name an API imported from another account [DOCUMENTED](#); that was not configured here.
- Leave out `external.api` and a mirror either copies nothing (`D7`) or silently copies a **local** stream of the same name (`D9a`). No error either way (`D7a`).

WHEN IS A DOMAIN VALID, AND WHAT DOES IT DO?

WHERE	DOMAIN SETTING	RESULT	EVIDENCE
one supercluster (gateway)	none	valid: one JetStream	A1 C2 F2
one supercluster (gateway)	one name everywhere	valid: one JetStream, named	DOCUMENTED
one supercluster (gateway)	different names	broken: one side blind, 10005	B1 – B6 G19 – G22
leaf, \$SYS not carried	a name per side	separate JetStreams, reached by API	D1 D8 H1 H11
leaf, \$SYS not carried	none	separate JetStreams	G1 G2
leaf, \$SYS carried, hub clustered	none	extends the hub's JetStream	DOCUMENTED
leaf, \$SYS carried	a name per side	separate JetStreams	DOCUMENTED

WHAT IT DOES GIVE YOU — USE A DOMAIN WHEN

- You need to **reach another JetStream by name**: `$JS.<domain>.API` , or `external.api` on a mirror (`D8` , `H11`).
- You want a leaf side to keep its own JetStream even if `$SYS` is carried [DOCUMENTED](#).

WHAT IT DOES NOT GIVE YOU

- Not a security boundary**. Anyone with the account can name the other API and read it, as `D8` and `H11` do on purpose.
- Different names inside one supercluster break it at start-up, and which side breaks is a coin toss (`B1` , `G19a`).

VERDICT

Accounts decide who may see what. Links and `$SYS`

decide who votes with whom. A domain names the result, and on a `$SYS` leaf it decides whether JetStream extends or stays separate. It is never a wall.

07

LINKS — T2 / A, THE GATEWAY

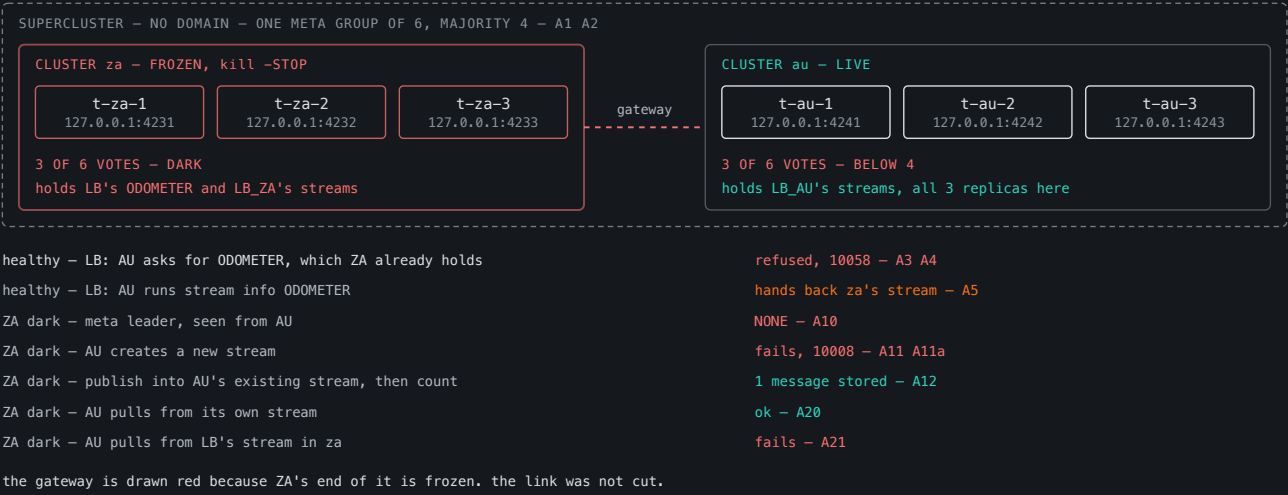
A gateway buys one name and sells your survival

A **gateway** joins two clusters into one **supercluster**. With JetStream on every server and no domain, the six servers form **one JetStream with one meta group**. One set of stream names per account, and one majority, held across the sea.

DECISION

Use a gateway when you want **one stream name per account, everywhere**, and you can add a third site that votes (card 08). Do not use a plain two-region gateway if either region must keep creating streams while the other is dark.

THE SHAPE, AND WHAT IT DOES WHEN ZA GOES DARK



08

QUORUM — T3 / C AND T4 / F, THE ARBITER

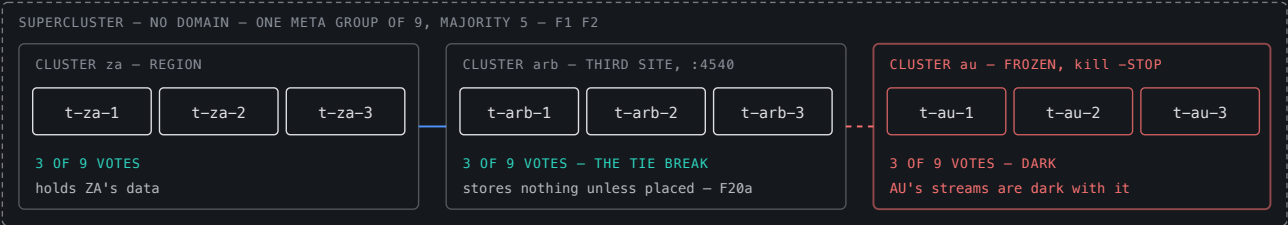
An arbiter site buys the vote back

Two regions of three cannot out-vote each other. **Three sites can.** Add a third cluster on the same gateway that votes and stores almost nothing. The meta group grows to 9, and either region alone can go dark.

DECISION

If you want one stream name across regions **and** creates during a region outage, pay for a third site. A three-node arbiter is the safe size. Keep clients off it.

T4 — NINE SERVERS, THREE SITES, ONE META GROUP — AND AU GOES DARK



AU dark — 6 of 9 left: is a leader elected?
AU dark — ZA creates a new 3-replica stream
the other way — ZA dark: AU elects, creates, stores
AU dark + one arbiter node dark — 5 of 9, exactly 5
one more node dark — 4 of 9
4 of 9 — publish into an existing stream, then count
after each recovery — meta group size

the gateway to au is drawn red because au is frozen. the link was not cut.

yes — F3
ok — F4
yes / ok / 1 — F22 F23 F24
still elects and creates — F5 F6
NONE; create fails — F7 F8
1 message stored — F9 F21
9 — F10 F25

The middle box is the purchase. It stores nothing and it is why the left box can still create.

MECHANISM

- › The meta group counts **servers**, not sites (card 02). Nine need five. Either region dark leaves six (F3 , F22).
- › The arbiter only has to be reachable over the gateway and vote. It holds no copy, KV or consumer unless one is placed on it (F20a , C20a). Card 04.
- › A stream keeps its own group. Even with no meta leader, a publish into an existing stream is stored while that stream's own group keeps its majority (F9 , F21). Card 03.
- › The arbiter does not copy data across regions. Every replica of a stream stays in one cluster (F17). So a stream stored in the dark region is dark too, even while the vote lives (F28). The region's own streams still read (F27). Card 03.

ONE NODE OR THREE — BOTH MEASURED

	T3 / C — 1-NODE ARBITER	T4 / F — 3-NODE ARBITER
meta group, majority	7, majority 4 (C1 , C2)	9, majority 5 (F1 , F2)
AU dark: ZA creates	ok C4	ok F4
ZA dark: AU creates	ok C6	ok F23
slack with a region dark	none — 4 of 7 (C12)	one node — 6 of 9 (F11 , F5)
stores data if told to	yes, R1 only (C8 , C10); R3 fails (C9)	not tested INFERRED: yes, R3 fits

WHAT IT DOES GIVE YOU

- › **One stream name per account, and creates with either region dark.** The only shape here that gives both while keeping one shared JetStream namespace. T5 also creates with either region dark, but gives each region its own namespace (card 09).
- › The third site is small. It is a vote, not a data centre (C20a).
- › Recovery is automatic (F10 , F25).

WHAT IT DOES NOT GIVE YOU

- › **No read of the dark region's data.** With ZA dark and the vote alive, AU cannot read a stream stored in ZA (F28). It gets a timeout, not 10008 (F28a). The same read worked before the freeze (F26). A copy in the healthy region needs a mirror (card 10).
- › **You need a real third place.** Three zones in one region do not survive that region's outage INFERRED .
- › The meta group spans the WAN, so every create waits for it INFERRED . Latency was not measured.
- › A one-node arbiter has no slack (C12). A client on its port stores data there (C10 , C11).

VERDICT

Buy the third site, or give up one stream name across regions. Without a third site, card 07's freeze stays.

09

LINKS — T5 / D AND H, HUB AND LEAF

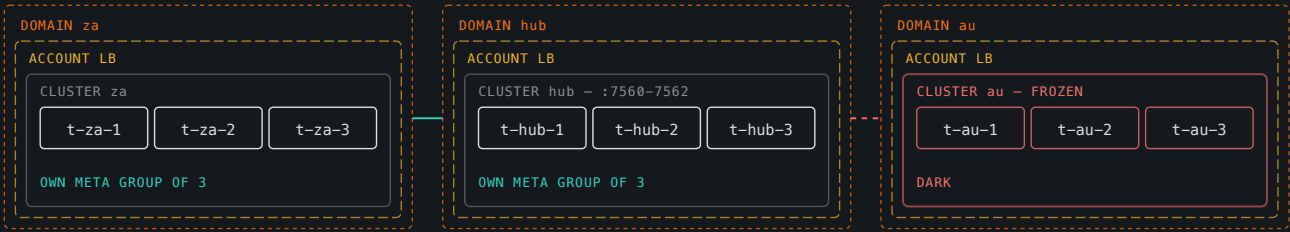
This lab's leaf link gives each region a vote, and a double copy

Each region hangs off a 3-node hub on a **leafnode link**. As this lab builds it, `$SYS` is **not** carried over the leaf and each side sets its own domain. The result is **three JetStreams**, each with its own meta group of 3.

DECISION

Use this shape when **each region must keep creating streams alone**. Accept that one stream name no longer means one stream, and write every shared copy down on purpose.

T5 — THREE JETSTREAMS, AND WHAT THEY DO WHEN AU GOES DARK



each leaf remote binds LB only. `$SYS` is not carried over the leaf.

healthy — meta groups, and their sizes

LB — both regions create ODOMETER

LB — one publish in ZA, then count both

AU dark — ZA: leader, create, store

whole hub dark — both regions create and store

mirror of an au stream, name only

the same, with `external.api $JS.au.API`

LB_ZA, LB_AU — no leaf remote on T5

3; 3 / 3 / 3 — D1 D2
ok, in za / au, no 10058 — D3 D4
1 / 1 STORED, TWO copies — D5 D5a
yes / ok / 1 — D24 D25 D26
ok — D10–D13 D23
0 copied, no error — D7 D7a
2 copied — D8
six islands — D21 D22 D22a

Three orange boxes, three votes. And one account box in each, so one send is stored twice.

MECHANISM

- Two settings made three JetStreams here: `$SYS` is not carried over the leaf, and each side sets a domain (`D1`). With no domain at all the hub still stayed separate (`G1` , `G2`), so the `$SYS` setting did the separating **INFERRED** . Card 06.
- This is not the only way a leaf can work. A leaf that carries `$SYS` and sets no domain extends a clustered hub's JetStream instead **DOCUMENTED** . Not tested here.
- Each region's meta group is 3 servers at home, quorum 2 (`D2`). Another site going dark changes nothing (`D24` – `D26` , `D10` – `D13`).
- A leaf remote binds **one** account (`D22a`). The lab bound `LB` , so its subjects cross the link, and each JetStream stores its own copy (`D5`). Nobody can refuse the name (`D3` , `D6`).
- A stream in another domain is reached through its API: `external.api = $JS.au.API` (`D8`). Without it a mirror copies nothing, or silently copies a **local** stream of the same name (`D7a` , `D9a`).

VARIANT H — THE SAME SHAPE, AN ACCOUNT PER REGION

- A second leaf remote per region server binds `LB_ZA` or `LB_AU` to the hub. Three JetStreams still (`H1`). One publish in `LB_ZA` reads `1 / 0` (`H6`). The hub holds three namespaces, not one shared one (`H9a`). A hub stream on the same subject still captures the live publish (`H8`): H stops ZA / AU cross-capture, not capture at the hub.
- With the whole hub gone, a region still creates and stores (`H12` – `H14` , `H16`). **A region dark was not run for H**. What H buys is a live subject feed to the hub, and nothing more (`H15`).

WHAT IT DOES GIVE YOU

- Each region creates and stores alone** (`D25` , `D26`).
- The hub is a reader, not a dependency. Kill it and both regions carry on (`D10` – `D13` , `H14a`).
- No shared majority, so a region can be added without a re-vote **INFERRED** .

WHAT IT DOES NOT GIVE YOU

- A shared account over this leaf captures twice, silently** (`D5` , `D6`).
- That double capture is **live only**, not replication. A message sent while the link is down reaches one side, and nothing catches up later. Subject capture is not outage recovery. Only a mirror copies history **INFERRED** .
- Nothing is shared until somebody writes it down (`D15`). A mirror is a real thing to build and watch (card 10).
- A name-only mirror can copy the wrong stream with no error (`D9a`).

VERDICT

For two regions that must each keep working, start here — with an account per region, and every shared stream a written mirror.

10

COPYING — MIRRORS

A mirror is the second copy — and it can copy the wrong stream

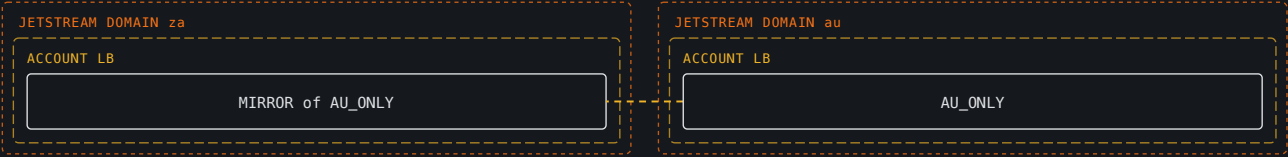
A mirror names a source stream. The name is looked up **in the asking side's own JetStream and own account**. If the source is anywhere else, the mirror must say where. If it does not, it is still accepted — and copies the wrong thing, or nothing.

DECISION

Every mirror whose source is in another JetStream sets `mirror.external.api`. Treat a mirror that has never been tested by counting as a **bug**, even when it is green.

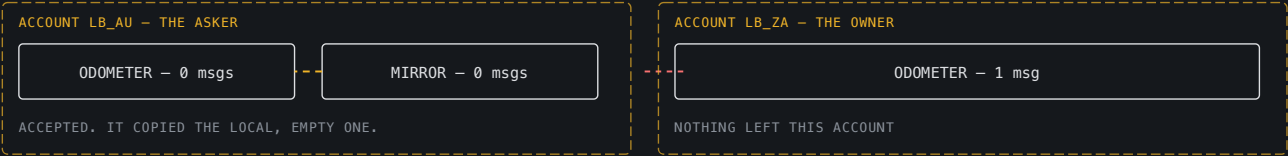
THE SAME CONFIG, ONE LINE APART

THE RIGHT WAY, ACROSS THIS LAB'S LEAF — NAME THE OTHER DOMAIN'S API



`external.api = $JS.au.API`: 2 copied — D8 (H11 the same, hub to za). without it: 0, no error — D7 D7a H10

THE TRAP — A NAME ALONE CANNOT CROSS AN ACCOUNT



ACCEPTED. IT COPIED THE LOCAL, EMPTY ONE.
the mirror was NOT refused, and copied 0 of 1 messages — E17 E18 E18a

over a gateway there is one JetStream, so a name alone is enough: copied, no `external.api` — A15 A16 A17a

Both drawings are a working mirror. **Only the top one is the mirror you meant.**

MECHANISM

- › A mirror is a stream that copies another stream, one way. The copy is read-only.
- › A bare source name is looked up **in the asking side**: its own JetStream and its own account ([H7a](#) , [E18a](#)). Card 05 is the same fact seen from the other end.
- › `mirror.external.api` tells the mirror which JetStream API to ask. In this lab it names a **domain** prefix ([H11a](#)). No gateway, no shared vote.
- › A **name-only** mirror cannot cross an **account** ([E17](#) , [E18](#)). NATS documents a cross-account mirror: the owner exports its JetStream API, the other account imports it, and the mirror names it in `external.api` [DOCUMENTED](#) ([docs: cross-account](#)). **Not configured or measured here.** The measured way across an account is to export the subject (card 11).
- › The lab proves each copy by **counting messages on both sides**, never by the mirror's status — the wrong mirror reports healthy too ([D7a](#) , [E18a](#)).

WHERE THE SOURCE IS, AND WHAT THE MIRROR NEEDS

THE SOURCE STREAM IS

THE MIRROR NEEDS

PROOF

in the same JetStream and account (gateway)

its name only

[A15](#) [A16](#) [A17](#) [A17a](#)

in another domain, over this lab's leaf

`external.api = $JS.<domain>.API`

[D8](#) [H11](#)

... and the name only is given

0 copied, or a local stream copied

[D7](#) [H10](#) [D9](#) [D9a](#)

in another account, name only

the wrong stream — copies the asker's own

[E17](#) [E18](#) [E18a](#)

in another account, API exported and imported

`external.api` = the imported API prefix

[DOCUMENTED](#) not tested

WHAT IT DOES GIVE YOU

- › **Sharing without merging.** Each region keeps its own vote and still gets the other's data ([D8](#)).
- › It is explicit. You can read the config and say what crosses.
- › The copy is local, so it can still be read when the source region is dark [INFERRED](#).

WHAT IT DOES NOT GIVE YOU

- › **The copy lags**, and it stops moving while the source is dark [DOCUMENTED](#). Lag was not measured here.
- › You pay for the data twice, and for the link.
- › A wrong mirror is **silently wrong**, not loudly broken ([E17](#) , [E18](#) , [D9a](#)).

THE ONE TEST THAT CATCHES IT

- › Publish into the **source** only. Then count the mirror. If the count does not move, the mirror is bound to the wrong stream.

VERDICT

A mirror is how a copy crosses JetStreams on

purpose. Name the source's API, and test it by counting, never by its status.

A15–A17a, D7–D9a, H10, H11, E17–E18a · Card 10

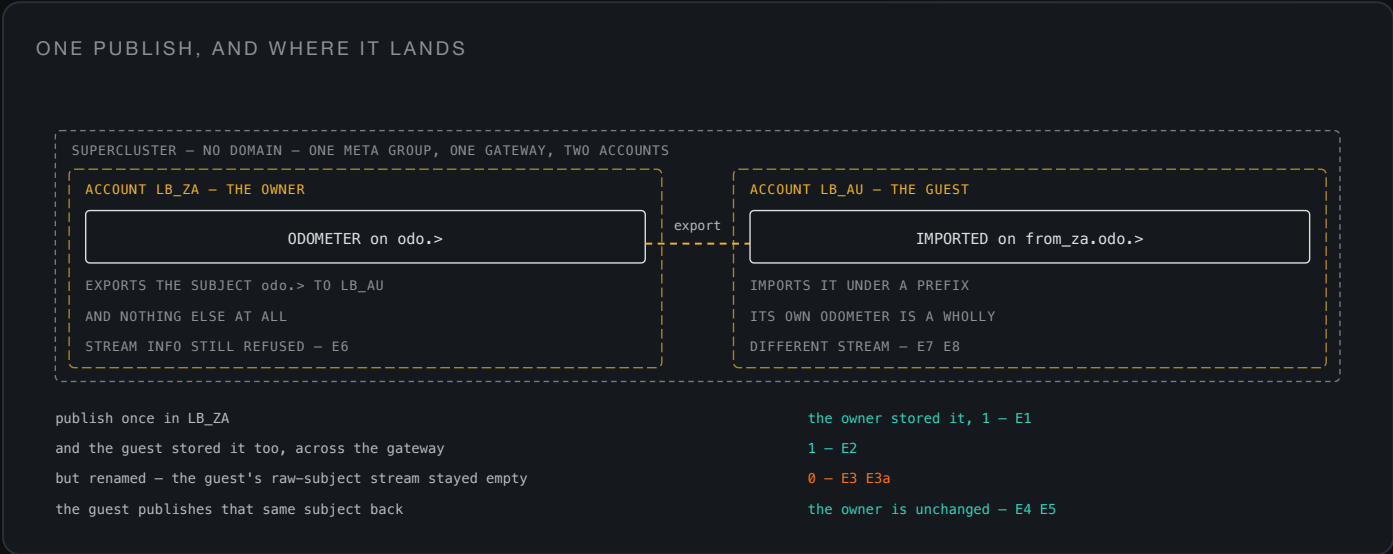
11

COPYING — T2 / E, EXPORT AND IMPORT

Export / import: the explicit way across an account

An account shares nothing by accident. To let one account see another's messages, the owner **exports** and the guest **imports** — two edits, in two places, by two people.

DECISION Share between tenants with **export / import and an import prefix**. Never by putting two tenants in one account, and never by hoping a domain will keep them apart (card 06).



12

TRAPS — T6 / G, BOTH LINKS AT ONCE

Both links at once is not both shapes at once

Add a hub leaf link to a gateway pair and it looks like belt and braces. It is not. **The gateway decides** (G24). You pay for a hub, and the two regions get no vote back.

DECISION

Pick **one** link type between the two regions. If they share a gateway, a leaf-linked hub cannot rescue them, however healthy it is.

A HEALTHY HUB, AND A REGION THAT STILL CANNOT CREATE

CLUSTER hub — NO DOMAIN — META GROUP OF 3

t-hub-1t-hub-2t-hub-3

leaf, binds LB · \$SYS not carried

SUPERCLUSTER — NO DOMAIN — ONE META GROUP OF 6 OVER BOTH REGIONS

CLUSTER za

t-za-1t-za-2t-za-3

3 OF 6 VOTES — BELOW 4

gateway

CLUSTER au — FROZEN, kill -STOP

t-au-1t-au-2t-au-3

3 OF 6 VOTES — DARK

healthy — meta groups; sizes hub / ZA / AU
LB — AU asks for ODOMETER too
LB — one publish in ZA, then count both
AU dark — ZA's meta leader
AU dark — ZA creates a new stream
AU dark — publish into ZA's existing stream, then count
AU dark — the hub keeps a leader and still creates
whole hub gone — ZA's group size; both regions create
phase 2 — add a domain per site (hub, za, au)
phase 2 — group size; place on the blind side; the hub

2; 3 / 6 / 6 — G1 G2 G3
refused, 10058 — G4
1 / 1 = ONE copy — G7 G7a
NONE — G10
fails, 10008 — G11 G11a
2 stored — G12
and lends ZA nothing — G13 G14 G14a
6; ok / ok — G16 G17 G18
1 of 2 regions elects — G19 G19a G20
still 6; 10005; hub elects — G21 G22 G23

The teal lines are real, and the blue line still decides. **A hub on a leaf has no vote in a gateway's meta group.**

MECHANISM

> The gateway still fuses **za** and **au** into one meta group of 6 (G2 , G3). The leaf links do not undo that.

> The hub stays its own JetStream of 3 with **no domain** set (G1). The leaf does not carry **\$SYS** , and a leaf joins a JetStream only when it does **DOCUMENTED** . Card 06.

> With AU dark, ZA holds 3 of 6 and cannot reach 4 (G10 , G11). The hub has a leader (G13), but it is not in that group, so it cannot lend a vote (G14a).

> Phase 2 adds a domain per site. It breaks the way variant B breaks: one region blind, still one group of 6, **10005** on the blind side (G19 — G22). The leaf does not rescue it. Only the hub, across the leaf, keeps its own leader (G23).

MOVING BETWEEN SHAPES — WHAT WAS MEASURED, AND WHAT WAS NOT

> **A leaf beside the gateway is coexistence, not a conversion.** It changes nothing (G24). A domain per site on the gateway breaks it (G19 — G22).

> **Switching T4 ↔ T5 in place is card 13.** The one procedure tested did not establish safe conversion. Other procedures are untested, so the choice is not one-way.

> If the goal is a third site that votes, join it with a **gateway**, not a leaf. That is card 08, and it is measured.

WHAT IT DOES GIVE YOU — THE SHORT LIST

> A hub gives **one more place to attach clients** that is not either region.

> The hub keeps a working JetStream of its own, and can die with no effect on either region (G16 — G18).

WHAT IT DOES NOT GIVE YOU

> **Every T5 property is gone.** **10058** is back (G4), one copy not two (G7a), and a dark region freezes creates (G11).

> You pay for a third site that **cannot lend a vote** (G14a).

> On a diagram it looks safer than a plain gateway. That is the trap.

13

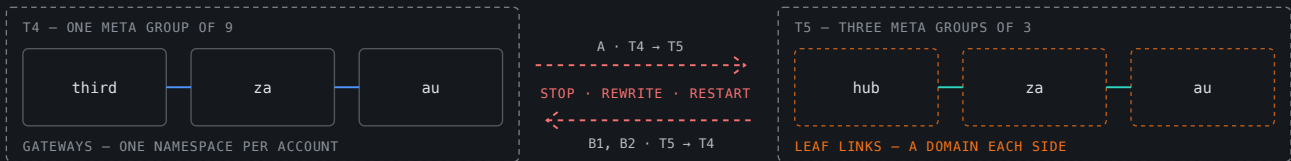
MOVING — T4 / S AND T5 / S, SWITCH IN PLACE · D03-R11 · D03-R12

The tested stop–rewrite–restart procedure did not establish safe conversion

One procedure was tested, both ways between T4 and T5: stop all nine servers, rewrite the configs, start them again on the same stores. It ran in three cases, on 2026-09-28 and again on 2026-09-29. **It failed in every case, both days.** This card is about that one procedure, not about every way to move.

DECISION Choose T4 or T5 **before the data lands**. If you must move later, do not rely on the tested procedure. Another way may work, but nobody has tested one.

THREE CASES, TWO RUNS, ONE PROCEDURE



case	2026-09-28 · judged afterwards	2026-09-29 · judged live
A · T4 → T5	T5 did not form data not read — unverified	T5 did not form — SA10 SA29 data held — SA11-SA20 SA30-SA39
B1 · T5 → T4, clean names	ZA streams gone AU held	1 group of 9, data held — SB10 SB29 1 placement unread — SB18
B2 · T5 → T4, one name in both regions	ZA streams unreadable ZA's name reads AU's data	2 of 3 ZA replicas removed — SC29 SC42 ZA's name reads AU's data — SC36 SC51
verdict, every case	failed	failed — SA41 SB41 SC59

The same procedure gave different results on two days. **A procedure you cannot repeat is not a procedure you can run on stores you need.**

MECHANISM — WHAT EACH CASE DID

- › **A · T4 → T5.** The servers kept the old membership. Every site still reported nine members, and no site elected a meta leader for minutes, after the switch and after a restart (SA7 , SA10 , SA28 , SA29). On 2026-09-29 each stream group still took acked publishes, and every message, reader position and KV key read back correct (SA11 – SA27). On 2026-09-28 the read needed a meta leader, so the data was not read: unverified, not disproved.
- › **B1 · T5 → T4, clean names.** One group of nine formed both times (SB10 , SB29). On 2026-09-28 the ZA streams could not be read and took no new writes. On 2026-09-29 every checked value matched. The one unmet check is a placement read that came back empty right after the switch; it read za after the restart (SB18 , SB37).
- › **B2 · T5 → T4, one name in both regions.** In T5 each region may own a stream called ODOMETER in account LB. In T4 the account has one namespace, so the two names collide. The ZA lookup resolved to AU's stream (SC36 , SC51). za-2, then za-1, removed their copies of every ZA stream; za-3 kept its copy. With one copy of three, those streams took no writes (SC29 , SC33 , SC35). After the restart no replica answered for two of them: **inconclusive** (SC42 , SC48).
- › **In both T5 → T4 cases, AU kept working** on both days. In case A on 2026-09-28, AU's data was not read either.

WHAT THIS CARD DOES NOT SAY

- › **It does not make T4 or T5 a one-way choice.** One procedure failed. Other procedures are untested. **PROPOSED** Build the new shape beside the old one, copy each stream across, move clients region by region, remove the old link last. **Nobody has run this.**
- › **It does not say why the ZA replicas were removed.** The cause is not proved.
- › **Three different things, and only two were seen.** *Replica removal* was seen: za-2 and za-1 removed their copies. *Unavailable data* was seen: those streams took no writes, and after the restart no replica answered for two of them (SC42 , SC48). *Irrecoverable loss* is neither proved nor disproved.
- › **A surviving copy proves nothing by itself.** Files remain on za-3. That does not show the data in them is whole, and it does not show they can be brought back. Neither was tested.
- › **One failure is the test's own.** The probe acked through the ZA name, so AU's reader moved on by one (SC38 , SC56). That is a side effect of the lab, not of NATS.
- › **Provenance.** The 2026-09-29 verdicts were judged live. The 2026-09-28 verdicts were derived from its evidence afterwards. The report's check IDs are the second run's.

WHAT HELD

- › **The rig checks.** All passed in every case: no checked setup or harness requirement failed. That does not make every failed check the conversion's. One came from the test probe (SC38 , SC56).
- › **AU**, and every stream stored there, in both T5 → T4 cases, both days.
- › On 2026-09-29, **the data in A and B1**: messages, reader positions and KV keys (SA11 – SA39 , SB11 – SB39).

WHAT BROKE

- › **T4 → T5 never reached T5**, on either day (SA10 , SA29).
- › **T5 → T4 is not repeatable.** In B1 the ZA streams were gone one day, and matched the next.
- › **A name in both regions collides** in T4 (SC36 , SC51). ZA's replicas were removed, and those streams became unavailable (SC29 , SC42).

VERDICT

Do not rely on the tested stop–rewrite–restart procedure to switch T4 and T5. Pick the shape first. Before any attempt to merge into T4, resolve same-name streams in each shared account — but unique names did not make it safe: B1 had clean names, and on 2026-09-28 its ZA streams were gone.

14

QUORUM — T4 / F, THE META LEADER IN THE HUB · D03-R13 — D03-R18

Hub placement is a request; regional recovery can change leadership

Hub leadership can be requested, but regional recovery can disturb it. Metadata quorum and stream availability remain separate. Measured on 2026-10-05 with `lab/10-hub-meta-leader.sh` ; every count is in `REPORT-10.md` , tied to a named run.

DECISION

If you want the meta leader in the hub, **ask for it with a step-down, and ask again after every regional recovery**. It is a request, not a setting. Plan a metadata outage and a stream outage as two different things.

MEASURED, RUN 10-20261005-150304 — META GROUP AND STREAMS, SHOWN APART

T4 — ONE META GROUP OF 9 · QUORUM 5 · NO DOMAIN SET

za · 3 voters

arb (hub) · 3 voters

au · 3 voters

state	healthy	one region dark	both regions dark	all nine back
META GROUP may change metadata	1 leader, 9 agree ML4	hub leader holds ML44 ML61	no leader, no create ML116 ML120	recovers — ML53 ML131 leader can move ML54 ML71 ML106
STREAMS may store a message	3 replicas each, write ML14–ML16	its stream: no ack ML140 ML151 others write — ML45 ML62	hub stream still writes ML118 ML119 ZA, AU: not probed	every message back ML55 ML133
hub step-down	--cluster arb put the leader in the hub, every trial — ML26 ML36 ML93 ML111			

A vote does not carry data, and data does not carry a vote. **The hub can keep writing with no meta leader, and a region can lose writes with a healthy meta group.**

MECHANISM — WHAT THE MACHINE DID

- Start-up does not put the leader in the hub. Nothing in the config pins it (ML5a). A step-down with `--cluster arb` moved it there every time it was asked (ML26 , ML36 , ML93 , ML111).
- One region dark leaves six voters, above the quorum of five. A hub leader held, in the same term, and creates went on (ML44 , ML49a , ML61 , ML66a). A leader in the dark region was replaced from outside it (ML78 , ML96).
- Both regions dark leaves three voters. The hub is healthy, but it has no meta leader and a create fails (ML116 , ML117 , ML120). Its own stream still takes writes and reads back (ML118 , ML119).
- On a return, recovery and stability are two answers. Every valid round recovered: peers current, one leader, creates work. Stability did not always hold. The leader left the hub — to the region that stayed up (ML54 , ML71) or to the returning one (ML106), or the same server came back in a higher term (ML157a).

WHAT THIS CARD DOES NOT SAY

- It does not say why a return disturbs leadership. The cause is not proved.
- It gives no rate. The rounds are few. The cross-run table in `REPORT-10.md` counts them per named run; quote that table, never a percentage.
- A timed-out publish is an unknown outcome, not a failure. The CLI gave up (ML140a , ML151a). In the cited run nothing was stored later (ML146a , ML157a), but in the earlier step-6 own-rig run one such message was. Why is not proved. Retry with the same `Nats-Msg-Id` .
- A leader field can be stale. After both regions went dark, the hub servers stopped naming the old leader one at a time (ML116a). Read every hub monitor, and wait.
- Provenance. The figure is run `10-20261005-150304` . Runs with a failed rig precondition count as inconclusive, never as a result.

WHAT HELD

- The request. A hub step-down put the leader in the hub, every trial (ML26 , ML111).
- One region dark, leader in the hub: the leader stayed, metadata and writes went on (ML44 , ML61).
- Recovery, in every valid round (ML53 , ML131).

WHAT BROKE

- Stability on return. The leader moved, or the term rose (ML54 , ML71 , ML106 , ML157a).
- The hub alone cannot change metadata. Both regions dark: no leader, no create (ML116 , ML120).
- A dark region's stream takes no write, even with a healthy meta group (ML140 , ML151).

VERDICT

Request hub leadership; do not depend on it. Re-request it after each regional recovery, and alert on where the meta leader is. Size each stream's replicas for the outage it must survive: the meta quorum will not do that for you.



CHOOSING

Pick the shape from the outage you must survive

Start from the sentence "during a region outage we must still be able to ...". The shape falls out of it. Nothing else in this deck decides for you.

THE SHAPES MEASURED, SIDE BY SIDE					
SHAPE	META GROUPS	A STREAM NAME IS UNIQUE IN	REGION ALONE CAN STILL CREATE?	SITES	CARD
T1 no link	2, of 3 each	each region, each account	yes — nothing shared	2	01
T2 / A gateway	1, of 6	the account, everywhere	no — 3 of 6	2	07
T3 / C + 1-node arbiter	1, of 7	the account, everywhere	yes, no slack	3, one tiny	08
T4 / F + 3-node arbiter	1, of 9	the account, everywhere	yes, both ways	3	08
T5 / D hub and leaf	3, of 3 each	each domain, each account	yes	2 + a hub	09
T5 / H + an account per region	3, of 3 each	each domain, each account	hub gone: yes. region dark: not run	2 + a hub	09
T6 / G gateway AND leaf	2, of 3 and 6	the account, over both regions	no — and you paid for a hub	3	12

CHOOSE THIS LAB'S LEAF SHAPE (CARD 09) WHEN

- > A region must keep taking work while the other is dark. This is the usual answer for logistics.
- > You accept a mirror per shared stream, and can name what is shared (card 10).
- > You have no credible third site, or do not want to pay for one.

CHOOSE THE ARBITER SHAPE (CARD 08) WHEN

- > One stream name per account, everywhere, is worth more than regional independence. T3 and T4 both keep that shared namespace and still create with either region dark (C4 , C6 , F4 , F23). T4 also survives one more server lost.
- > You have a real third location, not a third zone in one region.
- > You can live with every stream change waiting on the WAN **INFERRED**.

RULES THAT HOLD WHICHEVER YOU PICK

- > **Say which of the three writes you mean** before agreeing any of this (card 03).
- > **Accounts for tenants, always.** A stream name is unique per meta group and account (card 05). A domain is not a wall (card 06).
- > **Cross an account only with export / import**, with a prefix (card 11). A name-only mirror cannot do it and will not say so (card 10). A prefix is a namespace, not proof of origin.
- > **Choose before the data lands.** The one in-place T4 ↔ T5 procedure tested has not shown safe, repeatable operation. Other ways are untested; the choice is not one-way (card 13).
- > **Call a failover ready** only when a real operation succeeds, not when /jsz names a leader (card 02).
- > **Never add a leaf hub to a gateway pair** expecting quorum (card 12).
- > **Request the meta leader in the hub; do not depend on it.** Regional recovery can change leadership. Plan metadata and stream outages apart (card 14).

§

PROVENANCE

Where every number came from

A figure with no provenance becomes a stale constant. Every measured number in this deck comes from one generated report, `REPORT.md`, built by one rig on one machine.

THE RUN	
WHAT	VALUE
When	2026-09-28 13:40 SAST — the nine topologies
T4 ↔ T5 switch	2026-09-28 and 2026-09-29 09:39 SAST, <code>lab/09-switch-t4-t5.sh</code> ; the report's IDs are the second run's
Exercise 10, hub meta-leader	2026-10-05, <code>lab/10-hub-meta-leader.sh</code> on T4; figure from run <code>10-20261005-150304</code> . Its own report: <code>REPORT-10.md</code> and <code>REPORT-10.html</code> , generated by <code>lab/render-report-10.py</code> , with every kept run named
Server	nats-server v2.14.6
Client	nats CLI 0.4.0
Machine	Darwin 25.4.0 arm64, all servers on one host
Result	196 rig checks passed, 0 failed, 71 recorded observations
Procedure	83 procedure checks met, 18 not met; 3 procedure verdicts, all failed (<code>SA41</code> , <code>SB41</code> , <code>SC59</code>). 09-29 verdicts ran live; 09-28 verdicts derived afterwards.
Produced by	<code> demos/03-multi-cluster-and-accounts/lab/run-all.sh </code>
Written to	<code>REPORT.md</code> and <code>REPORT.html</code> — both generated, never hand-edited

WHICH CARD USED WHICH CHECKS		
CARD	SHAPE	CHECKS CITED
01 the map	all	<code>T1a – T1c</code> <code>A1</code> <code>C1</code> <code>F1</code> <code>D1</code> <code>G1</code>
02 Raft and quorum	T1, T2, T3, T4, T5	<code>A2</code> <code>A10</code> <code>C1</code> <code>C12</code> <code>F1 – F11</code> <code>D2</code> <code>D24</code> <code>G10</code> <code>T1g</code>
03 three questions	T2 / A, and all	<code>A11 – A21a</code> <code>C4</code> <code>C6</code> <code>D23 – D26</code> <code>F4</code> <code>F21 – F28a</code> <code>G11</code> <code>G12</code> <code>H16</code> <code>T1h</code>
04 placement	T4 / F, T3 / C	<code>F13 – F20a</code> <code>C7 – C18</code> <code>A9</code> <code>A15</code> <code>A19</code> <code>A26</code> <code>B4</code> <code>E13</code> <code>E14</code>
05 accounts and names	T2, T5, E	<code>A4 – A8</code> <code>D3 – D6</code> <code>D18</code> <code>H6</code> <code>G4</code> <code>G7</code> <code>E7</code> <code>E8</code> <code>T1d</code> <code>T1f</code>
06 domains	B, T5 / D, H, G	<code>B1 – B6</code> <code>D1</code> <code>D7 – D9a</code> <code>H1</code> <code>H7</code> <code>H11</code> <code>G1</code> <code>G19</code> <code>G22</code>
07 gateway	T2 / A	<code>A1 – A25</code> <code>E13</code> <code>E14</code>
08 arbiter	T3 / C, T4 / F	<code>C1 – C12</code> <code>C20a</code> <code>F1 – F28a</code>
09 leaf link	T5 / D, T5 / H	<code>D1 – D26</code> <code>H1 – H16</code> <code>G1</code> <code>G2</code>
10 mirrors	T2, T5, H, E	<code>A15 – A17a</code> <code>D7 – D9a</code> <code>H10</code> <code>H11</code> <code>E17 – E18a</code>
11 export / import	T2 / E	<code>E1 – E8</code>
12 both links	T6 / G	<code>G1 – G24</code>
13 switch in place	T4 / S, T5 / S	<code>SA7 – SA41</code> <code>SB10 – SB41</code> <code>SC29 – SC59</code>
14 hub meta-leader	T4 / F, ex 10	<code>ML4 – ML19</code> <code>ML26</code> <code>ML44 – ML71</code> <code>ML93 – ML111</code> <code>ML116 – ML120</code> <code>ML131 – ML157a</code>

One machine, three days: 2026-09-28, 2026-09-29 and 2026-10-05. Re-run the rig before quoting any of it elsewhere.

FOUR KINDS OF CLAIM, AND HOW EACH IS MARKED	
MARK	MEANS
a check ID, such as <code>A11</code>	Measured in the run above. The row in <code>REPORT.md</code> holds the expected and the actual value.
<code>DOCUMENTED</code>	Said by the NATS docs listed below. Not run in this lab.
<code>INFERRED</code>	Follows from measured facts and the docs, but nobody ran it. Treat it as a question for a later run.
<code>PROPOSED</code>	A step we suggest. Nobody has run it. It is a plan, not a finding.

THE DOCS BEHIND EVERY “DOCUMENTED” MARK — DOCS.NATS.IO	
PAGE	USED FOR
Learn › topologies › Leaf nodes	a domain can stop JetStream extending across a <code>\$SYS</code> leaf; it is still not a security boundary
Leaf nodes › JetStream on leaf nodes	a leaf that carries <code>\$SYS</code> and sets no domain extends a clustered hub's JetStream; a domain on the leaf side keeps it separate; <code>\$JS.<domain>.API</code> reaches another domain
Clustering › JetStream clustering	one meta group; each stream and each consumer is its own Raft group; quorum is more than half; replicas 1, 3 or 5
Concepts › JetStream › Streams	placement by cluster or by tags (tags not tested here); mirrors and <code>external.api</code>
Reference › config › leafnodes › remotes	a leaf remote binds one local account; <code>urls</code> is failover for one link
Reference › JetStream › Cross-account subjects	a mirror or source across accounts, through an exported JetStream API (card 10, not tested here)

- WHAT THESE NUMBERS ARE NOT
- › **No timing in this deck is a production figure.** Every server ran on one laptop with no WAN between them. Latency and mirror lag were not measured (`D03-R7`).
 - › **Every outage is `kill -STOP` on a whole region.** No split brain, no packet loss, no real WAN partition.
 - › **A check ID is permanent.** IDs are never reused or renumbered, so a card citing `A11` keeps meaning the same measurement across runs.
 - › If a card and `REPORT.md` disagree, the report is right. Re-run `lab/run-all.sh` and correct the card.